https://chatgpt.com/share/69a51cfc-1fe4-8012-b71b-ded691244ad6

Frameworks used:

- Flask (Backend)
- Jinja2 + HTML + Bootstrap (Frontend)
- SQLite (Programmatically created)
- No JavaScript for core logic

✅ 1️⃣ PROJECT FOLDER STRUCTURE

Create a folder:

```
placement_portal/
│
├── app.py
├── database.db (auto created)
│
├── templates/
│   ├── base.html
│   ├── login.html
│   ├── register_student.html
│   ├── register_company.html
│   ├── admin_dashboard.html
│   ├── company_dashboard.html
│   ├── student_dashboard.html
│   └── create_drive.html
│
└── static/
```

✅ 2️⃣ INSTALL REQUIREMENTS

Open terminal inside project folder:

```
pip install flask flask_sqlalchemy flask_login
```

✅ 3️⃣ MAIN BACKEND FILE (app.py)

Create app.py

```
from flask import Flask, render_template, redirect, url_for, request, flash
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, UserMixin, login_user, login_required,
logout_user, current_user
from werkzeug.security import generate_password_hash, check_password_hash
from datetime import datetime

app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret123'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database.db'

db = SQLAlchemy(app)
login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = "login"

# ------------------ MODELS ------------------

class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    role = db.Column(db.String(20))  # admin / student / company
    name = db.Column(db.String(100))
    email = db.Column(db.String(100), unique=True)
    password = db.Column(db.String(200))
    approved = db.Column(db.Boolean, default=False)

class Drive(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    job_title = db.Column(db.String(100))
    description = db.Column(db.String(200))
    deadline = db.Column(db.String(50))
    status = db.Column(db.String(20), default="Pending")
    company_id = db.Column(db.Integer, db.ForeignKey('user.id'))

class Application(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    student_id = db.Column(db.Integer, db.ForeignKey('user.id'))
    drive_id = db.Column(db.Integer, db.ForeignKey('drive.id'))
```

```
    status = db.Column(db.String(20), default="Applied")

# ------------------ LOGIN MANAGER ------------------

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

# ------------------ CREATE DB ------------------

@app.before_first_request
def create_tables():
    db.create_all()
    admin = User.query.filter_by(role="admin").first()
    if not admin:
        admin = User(role="admin", name="Admin", email="admin@admin.com",
                     password=generate_password_hash("admin123"), approved=True)
        db.session.add(admin)
        db.session.commit()

# ------------------ ROUTES ------------------

@app.route('/')
def home():
    return redirect(url_for('login'))

# ---------- LOGIN ----------

@app.route('/login', methods=['GET','POST'])
def login():
    if request.method == "POST":
        email = request.form['email']
        password = request.form['password']
        user = User.query.filter_by(email=email).first()

        if user and check_password_hash(user.password, password):
            if user.role != "admin" and not user.approved:
                flash("Waiting for admin approval")
                return redirect(url_for('login'))

            login_user(user)
```

```
        if user.role == "admin":
            return redirect(url_for('admin_dashboard'))
        elif user.role == "company":
            return redirect(url_for('company_dashboard'))
        else:
            return redirect(url_for('student_dashboard'))
    else:
        flash("Invalid credentials")

    return render_template("login.html")

# ---------- LOGOUT ----------

@app.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('login'))

# ---------- STUDENT REGISTER ----------

@app.route('/register_student', methods=['GET','POST'])
def register_student():
    if request.method == "POST":
        name = request.form['name']
        email = request.form['email']
        password = generate_password_hash(request.form['password'])

        student = User(role="student", name=name, email=email,
                       password=password, approved=True)

        db.session.add(student)
        db.session.commit()
        return redirect(url_for('login'))

    return render_template("register_student.html")

# ---------- COMPANY REGISTER ----------

@app.route('/register_company', methods=['GET','POST'])
```

```
def register_company():
    if request.method == "POST":
        name = request.form['name']
        email = request.form['email']
        password = generate_password_hash(request.form['password'])

        company = User(role="company", name=name,
                       email=email, password=password, approved=False)

        db.session.add(company)
        db.session.commit()
        flash("Waiting for admin approval")
        return redirect(url_for('login'))

    return render_template("register_company.html")

# ---------- ADMIN DASHBOARD ----------

@app.route('/admin')
@login_required
def admin_dashboard():
    if current_user.role != "admin":
        return redirect(url_for('login'))

    students = User.query.filter_by(role="student").count()
    companies = User.query.filter_by(role="company").count()
    drives = Drive.query.count()
    applications = Application.query.count()

    pending_companies = User.query.filter_by(role="company", approved=False).all()

    return render_template("admin_dashboard.html",
                           students=students,
                           companies=companies,
                           drives=drives,
                           applications=applications,
                           pending_companies=pending_companies)

@app.route('/approve_company/<int:id>')
@login_required
def approve_company(id):
```

```
    company = User.query.get(id)
    company.approved = True
    db.session.commit()
    return redirect(url_for('admin_dashboard'))

# ---------- COMPANY DASHBOARD ----------

@app.route('/company')
@login_required
def company_dashboard():
    if current_user.role != "company":
        return redirect(url_for('login'))

    drives = Drive.query.filter_by(company_id=current_user.id).all()
    return render_template("company_dashboard.html", drives=drives)

@app.route('/create_drive', methods=['GET','POST'])
@login_required
def create_drive():
    if request.method == "POST":
        title = request.form['title']
        desc = request.form['desc']
        deadline = request.form['deadline']

        drive = Drive(job_title=title,
                      description=desc,
                      deadline=deadline,
                      company_id=current_user.id)

        db.session.add(drive)
        db.session.commit()
        return redirect(url_for('company_dashboard'))

    return render_template("create_drive.html")

# ---------- STUDENT DASHBOARD ----------

@app.route('/student')
@login_required
def student_dashboard():
    if current_user.role != "student":
```

```
        return redirect(url_for('login'))

    drives = Drive.query.filter_by(status="Approved").all()
    return render_template("student_dashboard.html", drives=drives)

@app.route('/apply/<int:drive_id>')
@login_required
def apply(drive_id):
    existing = Application.query.filter_by(
        student_id=current_user.id,
        drive_id=drive_id
    ).first()

    if not existing:
        application = Application(student_id=current_user.id,
                                  drive_id=drive_id)
        db.session.add(application)
        db.session.commit()

    return redirect(url_for('student_dashboard'))

# ------------------ RUN ------------------

if __name__ == "__main__":
    app.run(debug=True)
```

✅ 🔼 BASIC HTML TEMPLATES

Create inside templates/

---

base.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Placement Portal</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-4">
  <a href="/logout" class="btn btn-danger float-end">Logout</a>
  <h2>Placement Portal</h2>
  <hr>
  {% block content %}{% endblock %}
</body>
</html>
```

---

login.html

```
{% extends 'base.html' %}
{% block content %}
<form method="POST">
  <input class="form-control" name="email" placeholder="Email"><br>
  <input class="form-control" type="password" name="password"
placeholder="Password"><br>
  <button class="btn btn-primary">Login</button>
</form>
<br>
<a href="/register_student">Register Student</a><br>
<a href="/register_company">Register Company</a>
{% endblock %}
```

---

register_student.html

```
{% extends 'base.html' %}
{% block content %}
<form method="POST">
  <input class="form-control" name="name" placeholder="Name"><br>
  <input class="form-control" name="email" placeholder="Email"><br>
  <input class="form-control" type="password" name="password"
placeholder="Password"><br>
  <button class="btn btn-success">Register</button>
</form>
{% endblock %}
```

---

register_company.html

(Same as student register)

---

admin_dashboard.html

```
{% extends 'base.html' %}
{% block content %}
<h4>Admin Dashboard</h4>
<p>Students: {{students}}</p>
<p>Companies: {{companies}}</p>
<p>Drives: {{drives}}</p>
<p>Applications: {{applications}}</p>

<h5>Pending Companies</h5>
{% for c in pending_companies %}
  {{c.name}}
  <a href="/approve_company/{{c.id}}" class="btn btn-sm btn-success">Approve</a>
  <br>
{% endfor %}
{% endblock %}
```

---

company_dashboard.html

```
{% extends 'base.html' %}
{% block content %}
<a href="/create_drive" class="btn btn-primary">Create Drive</a>
<h4>Your Drives</h4>
{% for d in drives %}
  {{d.job_title}} - {{d.status}} <br>
{% endfor %}
{% endblock %}
```

---

create_drive.html

```
{% extends 'base.html' %}
{% block content %}
<form method="POST">
  <input class="form-control" name="title" placeholder="Job Title"><br>
  <input class="form-control" name="desc" placeholder="Description"><br>
  <input class="form-control" name="deadline" placeholder="Deadline"><br>
  <button class="btn btn-success">Create</button>
</form>
{% endblock %}
```

---

student_dashboard.html

```
{% extends 'base.html' %}
{% block content %}
<h4>Available Drives</h4>
{% for d in drives %}
  {{d.job_title}}
  <a href="/apply/{{d.id}}" class="btn btn-sm btn-primary">Apply</a>
  <br>
{% endfor %}
{% endblock %}
```

✅ 5️⃣ HOW TO RUN THE PROJECT

1️⃣ Open terminal
2️⃣ Go inside folder:

```
cd placement_portal
```

3️⃣ Run:

```
python app.py
```

4️⃣ Open browser:

http://127.0.0.1:5000

✅ 6️⃣ DEFAULT ADMIN LOGIN

Email: admin@admin.com
Password: admin123

✅ 7️⃣ WHAT THIS VERSION COVERS

✓ Admin pre-created
✓ Company approval
✓ Student registration
✓ Login system
✓ Create placement drives
✓ Apply for drive
✓ Prevent duplicate applications
✓ SQLite created programmatically

If you want, I can next provide:

- ✅ Full upgraded version covering ALL evaluation requirements
- ✅ ER Diagram explanation for report
- ✅ Complete 5-page project report ready
- ✅ Viva questions & answers preparation
- ✅ Proper folder structure as per IITM format